Movimientos de inventario

Informe técnico y guía de demostración

El módulo permite consultar, crear, editar y eliminar entradas y salidas mediante la API existente. Se desarrolló sobre la carpeta con Bootstrap suministrada por el profesor, conservando sus rutas, la entidad Java y la configuración HTTP.

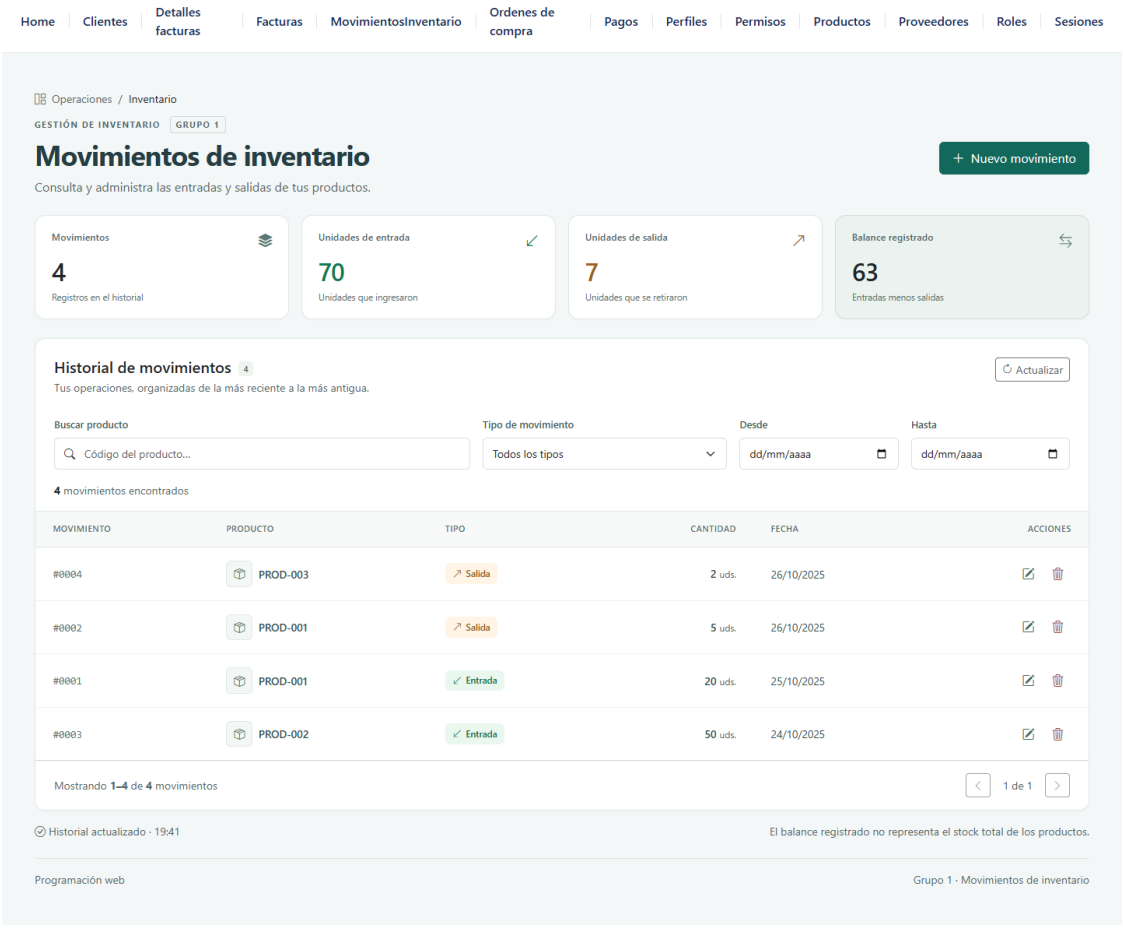
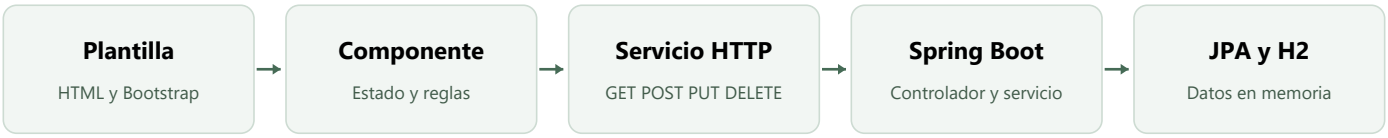


Figura 1 Vista general real con los cuatro registros de ejemplo y sus indicadores.

Entrega. Tres archivos del componente Angular y un servicio TypeScript. La documentación incluye una guía de sustentación, una explicación de los cuatro archivos línea por línea y capturas verificadas. Fecha de revisión: 10 de noviembre de 2026.

Arquitectura y comunicación

La plantilla presenta el estado y recibe los eventos. El componente valida y coordina; el servicio transforma esas decisiones en peticiones HTTP. Spring Boot atiende las rutas, el servicio Java usa el repositorio JPA y H2 conserva los registros mientras la aplicación permanece activa.



Al abrir la ruta **movimientosInventario**, ngOnInit llama a cargar(). La respuesta GET se guarda en movimientos. Los signals derivados calculan los indicadores y las filas visibles. Al guardar, la respuesta POST o PUT actualiza esa misma lista.

Operación	Petición utilizada	Respuesta satisfactoria
Consultar	GET /api/inventario-movimientos	200 y arreglo de movimientos
Crear	POST /api/inventario-movimientos	201 y movimiento con ID
Actualizar	PUT /api/inventario-movimientos/{id}	200 y movimiento actualizado
Eliminar	DELETE /api/inventario-movimientos/{id}	204 sin contenido

Ejemplo del cuerpo JSON de creación

```
{
  "productoCodigo": "PROD-001",
  "tipo": "ENTRADA",
  "cantidad": 12,
  "fecha": "2026-11-10"
}
```

El ID no se envía al crear. La API lo genera y lo devuelve. El backend también ofrece GET por ID; este módulo precarga la edición con los datos ya obtenidos del listado.

Datos y validaciones

Se respetaron los nombres y los tipos de la entidad InventarioMovimiento. La interfaz TypeScript describe el JSON esperado; las reglas del formulario atienden la interacción del usuario.

Campo	Representación	Regla del formulario
id	number / Integer	Generado por H2; no es un campo editable.
productoCodigo	string / texto de 40 caracteres	Obligatorio. Letras, números, guion o guion bajo. Máximo 40; se guarda en mayúsculas.
tipo	ENTRADA o SALIDA / String	Obligatorio. Solo se admiten esos dos valores.
cantidad	number / Integer	Entero positivo de 1 a 2147483647.
fecha	string / String	Obligatoria, formato AAAA-MM-DD y fecha existente.

Validar antes de llamar al servicio

guardar() marca todos los campos como tocados. Si el formulario es inválido, enfoca el primer control con error y regresa antes de enviar HTTP. Los atributos number, min, max y date ayudan a introducir datos; los validadores Angular comprueban las reglas del formulario.

Reglas de fechas y de cantidades

La fecha inicial se obtiene del reloj local del navegador. El validador de calendario comprueba el formato, la conversión a Date y que el día resultante coincida con el escrito. No se prohibieron fechas futuras porque la base recibida no define esa regla.

La cantidad tiene un patrón de entero positivo, un mínimo de uno y el máximo del Integer Java. Las pruebas comprobaron el rechazo de cero, números negativos, decimales y valores superiores al límite.

Alcance de estas reglas. Son validaciones del frontend. El backend del profesor no se modificó; una petición directa a la API puede omitirlas. La integridad de una aplicación en producción también debe imponerse en el servidor.

Ejecución e integración

Backend en IntelliJ

Abrir proyectoFinal/pom.xml como proyecto Maven y usar Java 17. En src/main/java/co/uis/proyecto/ProyectoApplication.java, ejecutar main. La aplicación debe indicar que Tomcat inició en el puerto 8080. Los recursos ya fueron instalados por el profesor.

Frontend en VS Code

Abrir proyectoFinal/web/proyecto-app-front. En su terminal ejecutar npm start y visitar http://localhost:4200/movimientosInventario. Si se prepara otro equipo, ejecutar npm ci antes de iniciar. Si ya existe un proceso en el puerto, usarlo o detenerlo antes de abrir otro.

Ubicación de los cuatro archivos

```
src/app/componentes/inventario-movimiento-component/
  inventario-movimiento-component.ts
  inventario-movimiento-component.html
  inventario-movimiento-component.css

src/app/services/
  inventario-movimiento-service.ts
```

La base ya registra la ruta, provideHttpClient(), los estilos y el bundle de Bootstrap y Bootstrap Icons. El ZIP de entrega no necesita incluir node_modules, Java, IntelliJ ni código de otros grupos.

Arranque por Maven en este equipo

La prueba JUnit de ejemplo del repositorio no tiene declaradas sus dependencias. Para arrancar la aplicación por consola manteniendo intacto ese POM, se utilizó el siguiente comando. Este arranque no representa una ejecución de las pruebas Java.

```
.\mvnw.cmd -Dmaven.test.skip=true spring-boot:run
```

La copia local también indica lang="es" en src/index.html para reconocer correctamente el idioma. El menú, los componentes de otros grupos y el código fuente Java se conservaron.

Si la API devuelve error 500, revisar la consola antes de modificar el frontend. En esta sesión se recuperó reiniciando Spring Boot y se comprobó HTTP 200 con cuatro registros. Reiniciar H2 en memoria vuelve a cargar data.sql.

Recorrido del código

Archivo	Responsabilidad
inventario-movimiento-service.ts	Contrato de datos y cuatro métodos HTTP con respuestas tipadas.
inventario-movimiento-component.ts	Estado, validaciones, filtrado, paginación y coordinación del CRUD.
inventario-movimiento-component.html	Interfaz, etiquetas, condiciones y eventos conectados con el componente.
inventario-movimiento-component.css	Personalización de Bootstrap, contraste, diálogos y adaptación móvil.

Estado y valores derivados

signal conserva valores como movimientos y guardando. computed deriva entradas, salidas, balance y filas visibles. Los filtros trabajan sobre la lista recibida, sin consultar de nuevo al servidor; los indicadores siempre usan el historial completo.

Guardar sin duplicar operaciones

En el componente, guardar comienza en la línea 232. Comprueba guardando, valida y prepara DatosMovimiento. editandoId igual a null selecciona crear; un ID selecciona actualizar. Los controles permanecen desactivados mientras la API responde.

Respuesta y limpieza

subscribe inicia la petición. next actualiza la lista después del éxito; error informa el problema y conserva los campos. finalize libera el indicador incluso cuando ocurre un error. takeUntilDestroyed cancela una petición pendiente al salir de la pantalla.

Identidad y eliminación

La edición usa map para sustituir por ID. El borrado usa filter para retirar solo el ID confirmado. La plantilla usa track m.id para identificar las filas aunque cambie su orden. Eliminar primero abre una confirmación con los datos del registro.

Codigo_explicado.html contiene los cuatro archivos completos, con líneas numeradas, explicación de cada línea, búsqueda y filtro por archivo. Las referencias corresponden a la misma versión incluida en el ZIP.

Formulario y tratamiento de errores

El formulario de creación y el de edición comparten los mismos controles. Los mensajes de validación indican qué se debe corregir. Cancelar no realiza cambios en la API.

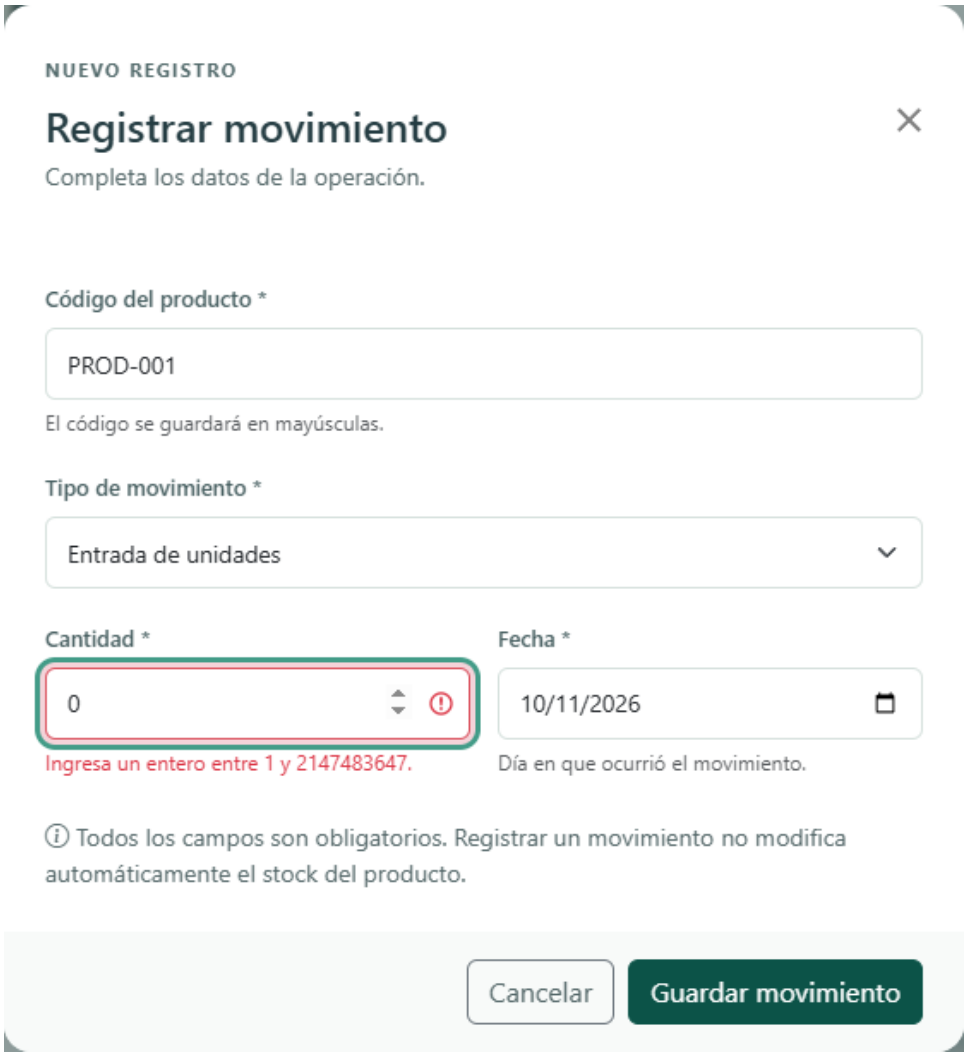


Figura 2 Captura ampliada real del rechazo de una cantidad igual a cero.

Si se pierde la conexión o la API rechaza una operación, el mensaje aparece en el listado, en el editor o en la confirmación según corresponda. Se liberan los botones y se conservan los valores del formulario para reintentar.

Los datos no cambian en pantalla hasta que el servidor confirma. No se presenta como eliminado un movimiento cuya petición DELETE haya fallado.

Bootstrap y accesibilidad

Bootstrap aporta la cuadrícula, los controles, las alertas, los botones y la tabla. Las tarjetas usan col-6 y col-xl-3: dos por fila en pantallas pequeñas y cuatro desde el punto de corte xl. Bootstrap Icons suministra los iconos locales, sin depender de un CDN durante la demostración.

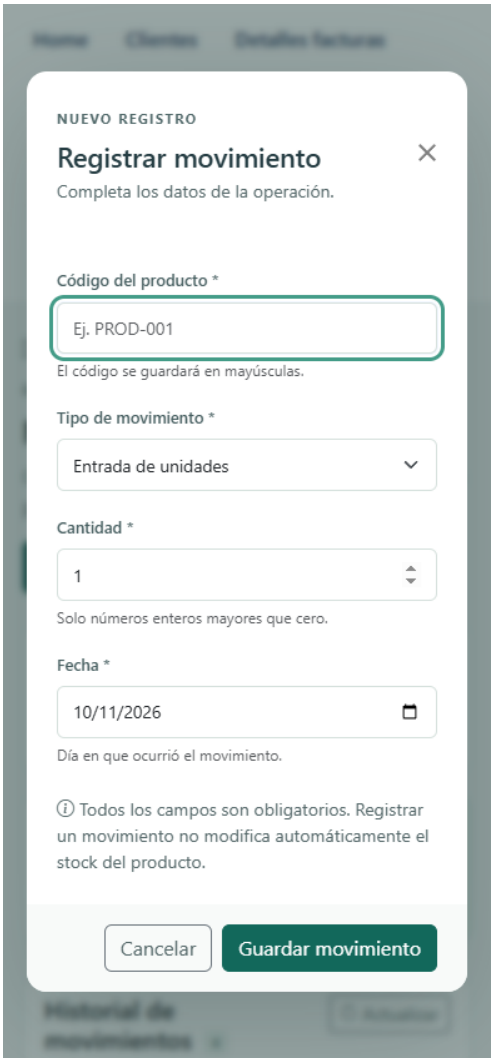


Figura 3 El formulario apila sus controles dentro del ancho de un teléfono.

La tabla usa table-responsive para conservar todas sus columnas y desplazarse dentro de su región. Los tipos se distinguen por texto e icono, además del color. Los campos tienen etiquetas asociadas y los botones de icono incluyen aria-label.

El diálogo nativo administra el foco, vuelve inerte el fondo y permite cancelar con Escape. Se protege el cierre durante una petición. Se revisaron reglas automáticas WCAG 2 A, 2 AA y 2.1 AA con axe-core en escritorio, formulario y móvil: cero infracciones detectadas en esas vistas.

Pruebas y evidencias

Se ejecutaron 26 comprobaciones en Chrome automatizado contra Angular y Spring Boot locales. Los casos de error y paginación usan respuestas simuladas únicamente dentro del navegador de pruebas. Los registros de prueba se identificaron con un prefijo propio y se retiraron al finalizar.

Caso	Comprobación	Resultado
1	Carga e indicadores contra la API real	Aprobado
2	Carga local de iconos Bootstrap	Aprobado
3	Búsqueda sin distinguir mayúsculas	Aprobado
4	Combinación de código y tipo	Aprobado
5	Fechas con extremos inclusivos	Aprobado
6	Rango de fechas invertido	Aprobado
7	Estado sin resultados y limpieza	Aprobado
8	Fecha local y foco inicial	Aprobado
9	Campos obligatorios sin POST	Aprobado
10	Código con espacios rechazado	Aprobado
11	Cantidad cero rechazada	Aprobado
12	Cantidad negativa rechazada	Aprobado
13	Cantidad decimal rechazada	Aprobado
14	Cantidad fuera de Integer rechazada	Aprobado
15	Fecha vacía rechazada	Aprobado
16	POST sin ID y normalización	Aprobado
17	Persistencia al recargar navegador	Aprobado
18	PUT conserva el ID y actualiza	Aprobado
19	Cancelar borrado conserva registro	Aprobado
20	DELETE confirmado retira registro	Aprobado
21	Error de guardado conserva formulario	Aprobado
22	Fallo de carga y recuperación	Aprobado
23	Paginación y reajuste de página	Aprobado
24	Escape y devolución del foco	Aprobado
25	Ancho móvil y diálogo adaptable	Aprobado
26	Consola limpia y originales intactos	Aprobado

La compilación de producción generó los archivos correctamente. Mantiene la advertencia del paquete inicial: 655,42 kB frente a 500 kB de aviso al incluir Bootstrap e iconos. No se superó el límite de error de 1 MB ni el presupuesto de estilos del componente.

Sustentación y demostración

Inicio y alcance

"Nuestro módulo registra entradas y salidas de inventario. Cada movimiento tiene un producto, un tipo, una cantidad y una fecha. El backend genera el ID. El balance resume el historial; no representa stock disponible".

Secuencia de demostración

1. Mostrar los cuatro registros de ejemplo y explicar 70 entradas, 7 salidas y balance de 63.

2. Buscar prod-001, combinar el tipo y las fechas, y limpiar los filtros. Los indicadores no cambian.

3. Intentar guardar con datos inválidos. Corregirlos y crear un movimiento de demostración con código único.

4. Editar ese registro para cambiar tipo y cantidad. Explicar PUT y la conservación del ID.

5. Recargar el navegador para mostrar que el dato se guardó en la API y no solo en un arreglo local.

6. Abrir la confirmación, cancelar y luego eliminar únicamente el dato de demostración.

7. Recorrer servicio, guardar(), formulario y plantilla; finalizar con la vista móvil y las pruebas.

Preguntas para ensayar

Por qué un servicio. Separa las peticiones HTTP de las decisiones y de la presentación. **Por qué subscribe.** Ejecuta la petición y atiende su resultado. **Por qué finalize.** Libera los indicadores en cualquier resultado.

Por qué computed. Deriva datos sin mantener copias manuales. **Por qué Omit.** Elimina el ID del cuerpo de creación. **Por qué el balance puede ser negativo.** No hay saldo inicial ni regla de disponibilidad en la API suministrada.

Guia_de_sustentacion.md amplía estas respuestas y presenta un recorrido de siete minutos. El grupo debe poder seguir un dato desde el campo hasta H2 y explicar el camino de vuelta a la tabla.

Límites y referencias

Contrato recibido del profesor

InventarioMovimiento guarda productoCodigo como texto y no como una relación JPA con Producto. Registrar o borrar un movimiento no modifica la propiedad stock de productos. Se conservó ese contrato porque la entrega solicitada son cuatro archivos del frontend.

Datos en memoria y origen permitido

H2 usa jdbc:h2:mem:proyecto_db. Recargar el navegador conserva los datos mientras el servidor está activo; reiniciar Spring Boot recrea la base y ejecuta data.sql. La configuración CORS permite http://localhost:4200, por lo que se utiliza ese origen y no su alternativa 127.0.0.1.

Herramientas y asistencia

Se usó la instalación de IntelliJ, Java, Node y Angular que dejó el profesor. La IA asistió el desarrollo, las comprobaciones y la documentación conforme al permiso del curso. La evaluación requiere que el Grupo 1 comprenda las decisiones y pueda demostrar su funcionamiento.

Documentación oficial de consulta

Formularios reactivos de Angular
https://angular.dev/guide/forms/reactive-forms

Signals de Angular
https://angular.dev/guide/signals

Peticiones HTTP de Angular
https://angular.dev/guide/http/making-requests

Cuadrícula de Bootstrap 5
https://getbootstrap.com/docs/5.3/layout/grid/

Biblioteca Bootstrap Icons
https://icons.getbootstrap.com/

Base técnica inspeccionada: entidad y controlador InventarioMovimiento, servicio Java y repositorio, application.properties, data.sql, app.routes.ts y app.config.ts. El componente de clientes suministrado se utilizó como referencia de integración.